

```

# -*- coding: utf-8 -*-
"""
Бот-калькулятор доставки сыпучих материалов.
Считает цену по формуле, ведёт диалог с клиентом, уведомляет владельца о заявке.
"""

import re
import logging
from telegram import Update
from telegram.ext import (
    Application, CommandHandler, MessageHandler, ContextTypes, filters,
    ConversationHandler
)

logging.basicConfig(level=logging.INFO)

# ===== НАСТРОЙКИ =====

BOT_TOKEN = "8396865150:AAFqAQV9kW6k0Lo5i-dkjnzgw4zmCxSiLE"

# ID чата владельца, куда будут приходить уведомления о заявках.
# Чтобы узнать свой chat_id: напиши боту @userinfobot в Telegram, он покажет твой
OWNER_CHAT_ID = 1368974175

# ===== ДАННЫЕ: ЦЕНЫ МАТЕРИАЛОВ (руб/тонна) =====
# ключ -> (цена, список ключевых слов для распознавания в сообщении клиента)

MATERIALS = {
    "Песок 0-2": {"price": 580, "keywords": ["песок 0.2", "песок 02", "пес
    "Песок 0-3": {"price": 540, "keywords": ["песок 0.3", "песок 03", "пес
    "Песок 0.3-0.4": {"price": 620, "keywords": ["песок 0.3-0.4", "песок 034"]}
    "Песок 0.4": {"price": 680, "keywords": ["песок 0.4", "песок 04"]},
    "Грунт полевой": {"price": 550, "keywords": ["полевой грунт", "грунт полев
    "Чернозём": {"price": 750, "keywords": ["чернозем", "чернозём", "грун
    "Гравийный местный": {"price": 700, "keywords": ["гравийный местный", "гравий
    # Гравий 3/20 – особый случай, два варианта цены, обрабатывается отдельно
}

GRAVEL_3_20 = {"с налётом": 1200, "чистый": 1550}
GRAVEL_KEYWORDS = ["гравий 3/20", "гравий 3-20", "гравий 320"]

# ===== НАЦЕНКА ПО РЕГЛАМЕНТУ (по тоннажу) =====
# функция возвращает наценку в рублях по категории материала и тоннажу

```

```

def reglament_markup(category: str, tons: float) -> int:
    if category == "песок":
        if tons <= 5:
            return 2000
        elif tons <= 14:
            return 3000
        else: # 15
            return 4000
    elif category == "гравий_3_20":
        if tons <= 9:
            return 3000
        elif tons <= 14:
            return 4000
        else:
            return 5000
    elif category == "гравийный_местный":
        if tons <= 4:
            return 2500
        elif tons <= 14:
            return 4000
        else:
            return 5000
    elif category == "грунт":
        if tons <= 4:
            return 2500
        elif tons <= 9:
            return 3000
        else: # 10-15
            return 4000
    return 3000 # запасной вариант, если категория не определена

def get_category(material_name: str) -> str:
    if "Песок" in material_name:
        return "песок"
    if "Гравийный" in material_name:
        return "гравийный_местный"
    if material_name in ("Грунт полевой", "Чернозём"):
        return "грунт"
    return "песок"

# ===== ДОСТАВКА ПО ПОСЁЛКАМ (склад Холмогоровка) =====
# TODO: добавить остальные посёлки и остальные 3 склада по мере оцифровки карт

DELIVERY_HOLMOGOROVKA = {
    "зеленоградск": 4500,
    "гурьевск": 3000,
    "калининград": 3000,

```

```

        "светлогорск": 5000,
    }

MIN_DELIVERY = 3000
MIN_DELIVERY_15T = 4500

def get_delivery_price(city: str, tons: float) -> int:
    city_key = city.lower().strip()
    price = DELIVERY_HOLMOGOROVKA.get(city_key, MIN_DELIVERY)
    floor = MIN_DELIVERY_15T if tons == 15 else MIN_DELIVERY
    return max(price, floor)

# ===== ФОРМУЛА РАСЧЁТА =====

LOADING = 400
VAT_MULTIPLIER = 1.14

def round_up_100(value: float) -> int:
    return int((value // 100 + (1 if value % 100 else 0)) * 100)

def calculate_price(material_price: int, tons: float, delivery: int, markup: int):
    subtotal = material_price * tons + LOADING + delivery + markup
    total = subtotal * VAT_MULTIPLIER
    return round_up_100(total)

# ===== РАСПОЗНАВАНИЕ СООБЩЕНИЯ КЛИЕНТА =====

def extract_tons(text: str) -> float:
    match = re.search(r"(\d+(?:[.]\d+)?)\s*(?:т|тон)", text.lower())
    if match:
        return float(match.group(1).replace(",", "."))
    return None

def extract_material(text: str):
    text_low = text.lower()
    for gk in GRAVEL_KEYWORDS:
        if gk in text_low:
            return "ГРЯВИЙ_3_20"
    for name, data in MATERIALS.items():
        for kw in data["keywords"]:
            if kw in text_low:
                return name
    return None

def extract_city(text: str) -> str:
    text_low = text.lower()
    for city in DELIVERY_HOLMOGOROVKA.keys():

```

```

        if city in text_low:
            return city
    return None

# ===== СОСТОЯНИЯ ДИАЛОГА =====

WAITING_DATE, WAITING_PHONE = range(2)

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    tons = extract_tons(text)
    material = extract_material(text)
    city = extract_city(text)

    if not tons or not material or not city:
        await update.message.reply_text(
            "Уточните, пожалуйста: материал, объём в тоннах и населённый пункт до  

            Например: «Песок 0.3, 5 тонн, Калининград»"
        )
        return ConversationHandler.END

    if tons < 1 or tons > 15:
        await update.message.reply_text(
            "Пока принимаем заказы объёмом от 1 до 15 тонн. Для большего объёма с
        )
        return ConversationHandler.END

    delivery = get_delivery_price(city, tons)

    if material == "ГРАВИЙ_3_20":
        price_dirty = GRAVEL_3_20["с налётом"]
        price_clean = GRAVEL_3_20["чистый"]
        markup = reglament_markup("гравий_3_20", tons)
        total_dirty = calculate_price(price_dirty, tons, delivery, markup)
        total_clean = calculate_price(price_clean, tons, delivery, markup)
        context.user_data["order"] = {
            "material": "Гравий 3/20 (уточнить вариант)",
            "tons": tons, "city": city,
            "price_dirty": total_dirty, "price_clean": total_clean,
        }
        await update.message.reply_text(
            f"Гравий 3/20, {tons:.0f} тонн, доставка в {city.capitalize()}: \n"
            f"– с налётом: {total_dirty}₽ \n"
            f"– чистый: {total_clean}₽ \n \n"
            f"Какой вариант вам подходит?"
        )
        return WAITING_DATE

```

```

material_price = MATERIALS[material]["price"]
category = get_category(material)
markup = reglament_markup(category, tons)
total = calculate_price(material_price, tons, delivery, markup)

context.user_data["order"] = {
    "material": material, "tons": tons, "city": city, "price": total,
}

await update.message.reply_text(
    f"{tons:.0f} тонн с доставкой к вам будет {total}₽. На когда вам нужно?"
)
return WAITING_DATE

async def handle_date(update: Update, context: ContextTypes.DEFAULT_TYPE):
    date_text = update.message.text
    context.user_data["order"]["date"] = date_text
    order = context.user_data["order"]

    price = order.get("price") or order.get("price_clean")

    await update.message.reply_text(
        f"Хорошо, благодарю что обратились! {date_text} машина будет у вас "
        f"в первой половине дня с 9:00 до 14:00.\n"
        f"{order['material']}, {order['tons']:.0f} тонн – {price}₽.\n\n"
        f"Оставьте, пожалуйста, номер телефона для оформления заявки."
    )
    return WAITING_PHONE

async def handle_phone(update: Update, context: ContextTypes.DEFAULT_TYPE):
    phone = update.message.text
    order = context.user_data["order"]
    price = order.get("price") or order.get("price_clean")

    await update.message.reply_text("Спасибо! Заявка принята, скоро свяжемся для

    notify_text = (
        f"НОВАЯ ЗАЯВКА\n"
        f"{order['date']} {order['city'].capitalize()} "
        f"{order['material']} {order['tons']:.0f}т – {price}₽\n"
        f"Тел: {phone}"
    )

    if OWNER_CHAT_ID:
        await context.bot.send_message(chat_id=OWNER_CHAT_ID, text=notify_text)
    else:

```

```

        logging.warning("OWNER_CHAT_ID не задан — уведомление не отправлено. Заяв

return ConversationHandler.END

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Диалог отменён.")
    return ConversationHandler.END

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "Здравствуйте! Напишите, какой материал и сколько тонн вам нужно и куда д
        "Например: «Песок 0.3, 5 тонн, Калининград»"
    )

def main():
    app = Application.builder().token(BOT_TOKEN).build()

    conv = ConversationHandler(
        entry_points=[MessageHandler(filters.TEXT & ~filters.COMMAND, handle_mess
        states={
            WAITING_DATE: [MessageHandler(filters.TEXT & ~filters.COMMAND, handle
            WAITING_PHONE: [MessageHandler(filters.TEXT & ~filters.COMMAND, handl
        },
        fallbacks=[CommandHandler("cancel", cancel)],
    )

    app.add_handler(CommandHandler("start", start))
    app.add_handler(conv)

    print("Бот запущен...")
    app.run_polling()

if __name__ == "__main__":
    main()

```